

REQUIREMENT

=====

Build an expense management application for our organization.

ARCHITECTURE

=====

3. ARCHITECTURE DESIGN

Architecture and Diagram Summary

- **Frontend Layer:** User interface for expense submission and viewing reports.
- **API Layer:** RESTful APIs for handling requests from the frontend and external systems.
- **Application Services Layer:** Business logic and workflow management.
- **Persistence Layer:** Data storage and retrieval using PostgreSQL.
- **Security Layer:** Authentication and authorization using Spring Security.
- **External Systems:** Integration with third-party services (not in scope for this version).

Code Design

- `com.example.expense.api`: REST controllers and request/response DTOs.
- `com.example.expense.service`: Service classes for business logic.
- `com.example.expense.repository`: JPA repository interfaces.
- `com.example.expense.model`: Entity classes mapped to database tables.
- `com.example.expense.security`: Security configuration and user details service.
- `com.example.expense.validation`: Custom validation annotations and validators.

Implementation Plan

Phase 1: Setup

Tasks:

- Initialize the project with Spring Boot
- Configure PostgreSQL database
- Set up Spring Security

Dependencies: None

Expected Outcome: Project setup with database and security configurations.

Phase 2: Database

Tasks:

- Design and create database schema
- Implement JPA entities
- Set up database migrations

Dependencies: Phase 1

Expected Outcome: Database schema ready with entities and migrations.

Phase 3: Domain**Tasks:**

- Define domain models
- Implement repositories
- Create service interfaces

Dependencies: Phase 2

Expected Outcome: Domain models, repositories, and service interfaces implemented.

Phase 4: Business Logic**Tasks:**

- Implement service classes
- Define approval workflow logic
- Implement validation rules

Dependencies: Phase 3

Expected Outcome: Business logic and validation implemented.

Phase 5: APIs**Tasks:**

- Implement REST controllers
- Map request/response DTOs
- Integrate with service layer

Dependencies: Phase 4

Expected Outcome: APIs for expense submission and report retrieval.

Phase 6: Testing and Deployment**Tasks:**

- Write unit tests for services and controllers
- Perform integration testing
- Prepare deployment scripts

Dependencies: Phase 5

Expected Outcome: Application tested and ready for deployment.

API DESIGN

=====

4. API DESIGN

Endpoints

M e t h o d	Path	P u r p o s e	Request Body	Response Body	S u c c e s s f u l l s	E r r o r S t a t u s s
P O S T	/expenses	S u b m i t a n e w e x p e n s e	{"employeeId":1,"amount":100,"description":"Lunch","date":"2023-10-01"}	{"id":1,"employeeId":1,"amount":100,"description":"Lunch","date":"2023-10-01","status":"SUBMITTED"}	200, 404, 500	400

M e t h o d	Path	P u r p o s e	Request Body	Response Body	S u c c e s s t a t u s	E r r o r S t a t u s
GET	/expenses	Retrieve all expenses	N/A	[{"id":1,"employeeId":1,"amount":100,"description":"Lunch","date":"2023-10-01","status":"SUBMITTED"}]	200	404, 500

M e t h o d	Path	P u r p o s e	Request Body	Response Body	S u c c e s s t a t u s	E r r o r S t a t u s
GET	/expenses/{id}	Retrieve view as specific expense by ID	N/A	{"id":1,"employeeId":1,"amount":100,"description":"Lunch","date":"2023-10-01","status":"SUBMITTED"}	200	404, 500

M e t h o d	Path	P u r p o s e	Request Body	Response Body	S u c c e s s t a t u s	E r r o r S t a t u s
P U T	/expenses/{id}	U p d a t e a n e x i s t i n g e x p e n s e	{"amount":150,"description":"Dinner","date":"2023-10-01"}	{"id":1,"employeeId":1,"amount":150,"description":"Dinner","date":"2023-10-01","status":"SUBMITTED"}	200, 404, 500	400

M e t h o d	Path	P u r p o s e	Request Body	Response Body	S u c c e s s t a t u s	E r r o r S t a t u s
D E L E T E	/expenses/{id}	D e l e t e a n e x p e n s e b y I D	N/A	{"message": "Expense deleted successfully"}	200	404, 500

M e t h o d	Path	P u r p o s e	Request Body	Response Body	S u c c e s s t a t u s	E r r o r S t a t u s
P O S T	/expenses/{id}/approve	Approve an expense by ID	N/A	{"id":1,"employeeId":1,"amount":150,"description":"Dinner","date":"2023-10-01","status":"APPROVED"}	200, 404, 500	400

M e t h o d	Path	P u r p o s e	Request Body	Response Body	S u c c e s s t a t u s	E r r o r S t a t u s
P O S T	/expenses/{id}/reject	Reject an expense by ID	N/A	{"id":1,"employeeId":1,"amount":150,"description":"Dinner","date":"2023-10-01","status":"REJECTED"}	200	404, 500

Validation and Error Handling

Field Level Validation Rules

- employeeId: Required, must be a positive integer.
- amount: Required, must be a positive number.
- description: Optional, max length 255 characters.
- date: Required, must be a valid date in YYYY-MM-DD format.

Standard Error Response Envelope

```

{
  "timestamp": "2023-10-01T12:34:56.789Z",
  "status": 400,
  "error": "Bad Request",
  "message": "Validation failed for object='expense'. Error count: 1",
  "errors": [
    {
      "codes": "NotNull.expense.amount|NotBlank.expense.amount",
      "arguments": [
        {
          "codes": "expense.amount",
          "defaultMessage": "must not be null"
        }
      ],
      "defaultMessage": "must not be null"
    }
  ]
}

```

Business Failures to HTTP Status Codes

Business Failure	HTTP Status Code
Validation error	400
Resource not found	404
Internal server error	500

DATA MODEL

=====

5. DATA MODEL

Model

Table	Column	Data Type	Nullabl e	Ke y	Default	Descriptio n
Employee	id	UUID	No	PK		Unique identifier for employee
	name	VARCHAR	No			Employee's name
	email	VARCHAR	No			Employee's email

Table	Column	Data Type	Nullabl e	Ke y	Default	Descriptio n
	created_at	TIMESTA MP	No		CURRENT_TIMESTA MP	Record creation ti mestamp
	updated_at	TIMESTA MP	No		CURRENT_TIMESTA MP	Record last update ti mestamp
Expense	id	UUID	No	PK		Unique identifier for expense
	employee_id	UUID	No	FK		Employee who submitted the expense
	amount	DECIMAL	No			Expense amount
	description	VARCHAR	No			Expense descriptio n
	submitted_at	TIMESTA MP	No		CURRENT_TIMESTA MP	Expense submissio n timesta mp
	status	VARCHAR	No		'PENDING'	Expense status
Expense_Appro val	id	UUID	No	PK		Unique identifier for approval
	expense_id	UUID	No	FK		Expense being approved
	approver_id	UUID	No	FK		Employee who approved the expense

Table	Column	Data Type	Nullabl e	Ke y	Default	Descriptio n
	approved_at	TIMESTA MP	Yes		NULL	Approval t imestamp
	approval_stat us	VARCHAR	No		'PENDING'	Approval status

ER Diagram

```
{
  "entities": [
    { "name": "Employee", "fields": ["id", "name", "email"] },
    { "name": "Expense", "fields": ["id", "employee_id", "amount", "description", "submitte  
r_id", "approved_at"] },
    { "name": "Expense_Approval", "fields": ["id", "expense_id", "approver_id", "approved_  
at"] },
  ],
  "relationships": [
    { "from": "Employee", "to": "Expense", "label": "1:N" },
    { "from": "Expense", "to": "Expense_Approval", "label": "1:1" },
    { "from": "Employee", "to": "Expense_Approval", "label": "N:N" }
  ]
}
```

Schema Script

```

CREATE TABLE Employee (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    name VARCHAR NOT NULL,
    email VARCHAR NOT NULL UNIQUE,
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE Expense (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    employee_id UUID NOT NULL,
    amount DECIMAL NOT NULL,
    description VARCHAR NOT NULL,
    submitted_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    status VARCHAR NOT NULL DEFAULT 'PENDING',
    FOREIGN KEY (employee_id) REFERENCES Employee(id)
);

CREATE TABLE Expense_Approval (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    expense_id UUID NOT NULL,
    approver_id UUID NOT NULL,
    approved_at TIMESTAMP,
    approval_status VARCHAR NOT NULL DEFAULT 'PENDING',
    FOREIGN KEY (expense_id) REFERENCES Expense(id),
    FOREIGN KEY (approver_id) REFERENCES Employee(id)
);

CREATE INDEX idx_expense_employee_id ON Expense(employee_id);
CREATE INDEX idx_expense_approval_expense_id ON Expense_Approval(expense_id);
CREATE INDEX idx_expense_approval_approver_id ON Expense_Approval(approver_id);

```

Constraints and Indexes

- **Primary Keys:** Ensures each record is unique.
- **Foreign Keys:** Ensures referential integrity between tables.
- **Unique Constraints:** Ensures email uniqueness in the Employee table.
- **Check Constraints:** Ensures valid status values in the Expense and Expense_Approval tables.
- **Indexes:**
 - `idx_expense_employee_id`: Improves query performance on the `employee_id` column in the Expense table.
 - `idx_expense_approval_expense_id`: Improves query performance on the `expense_id` column in the Expense_Approval table.
 - `idx_expense_approval_approver_id`: Improves query performance on the `approver_id` column in the Expense_Approval table.

DOCUMENTATION WITH REMAINING PLACEHOLDERS

=====

API Documentation

Overview

Base URL: `http://localhost:5000` (development) | `https://api.example.com` (production)

Response Format: JSON

Authentication: Bearer token - reference security considerations

Rate Limits: 100 requests per minute

Authentication

Bearer Token

1. Obtain a token by logging in via the `/api/auth/login` endpoint.
2. Include in request headers: `Authorization: Bearer <token>`.
3. Token expires in 1 hour; refresh using the `/api/auth/refresh` endpoint.

Example Request with Auth

```
curl -X GET http://localhost:5000/api/expenses \
  -H "Authorization: Bearer your_token_here" \
  -H "Content-Type: application/json"
```

Endpoints

POST /api/expenses

Submit a new expense.

Request:

- **Method:** POST
- **Path:** `/api/expenses`
- **Headers:** `Authorization: Bearer <token>`, `Content-Type: application/json`
- **Request Body:**

```
{
  "employeeId": "1",
  "amount": 100,
  "description": "Lunch",
  "date": "2023-10-01"
}
```

Response:

- **Status 201 (Created):**

```
{
  "success": true,
  "data": {
    "id": "1",
    "employeeId": "1",
    "amount": 100,
    "description": "Lunch",
    "date": "2023-10-01",
    "status": "SUBMITTED"
  },
  "message": "Expense submitted successfully"
}
```

- **Status 400 (Bad Request):** Missing or invalid required field
- ****Status**